CS423 – CSC15003 – Software Testing (AI-augmented · 2026)
HOMEWORK – AI-FIRST EDITION (2026 v1.0)
HW02 – Domain Testing on EShop

Student Name: Lê Gia Bảo
Student ID: 23127325

I. Steps to Apply Domain Testing

Step 1: Analyze the requirements. The feature re**CS423 – CSC15003 – Software Testing (AI-augmented · 2026)**
HOMEWORK – AI-FIRST EDITION (2026 v1.0)
HW02 – Domain Testing on EShop

Student Name: Lê Gia Bảo
Student ID: 23127325

I. Steps to Apply Domain Testing

Step 1: Analyze the requirements. The feature requirements were reviewed to identify all user inputs, validation rules, business rules, constraints, and dependencies. This helps determine what inputs need to be tested.

Step 2: Identify the input domains. For each input, the valid and invalid domains were identified based on the specification. The data type, required format, acceptable values, and any dependencies between inputs were also determined.

Step 3: Partition the input domains into equivalence classes. Each input domain was divided into equivalence classes so that all values within the same class are expected to produce similar system behavior. Separate valid and invalid equivalence classes were created to represent different categories of input.

Step 4: Design representative test cases. At least one representative value was selected from every equivalence class. Additional test cases were added where necessary to cover important business rules, input dependencies, missing required fields, and combinations of invalid inputs.

Step 5: Review test coverage. The final set of test cases was reviewed to ensure that every equivalence class and every important validation rule was covered. Duplicate or redundant test cases were avoided while maintaining comprehensive coverage.

II. Steps to Apply Boundary Analysis

Step 1: Identify inputs suitable for Boundary Value Analysis. Each input was examined to determine whether it had measurable boundaries, such as numeric values, input length, or ordered ranges. Inputs without measurable boundaries, such as authentication status, enumerated values, or free-form text without length constraints, were excluded because Boundary Value Analysis is not applicable.

Step 2: Identify the boundary values. For each applicable input, the minimum and maximum boundaries were identified from the specification. Where only a minimum boundary existed, the analysis focused on values immediately below the minimum, at the minimum, immediately above the minimum, and a typical valid value. If both minimum and maximum boundaries existed, values around both boundaries were selected.

Step 3: Design boundary test cases. Test cases were created using the identified boundary values. While testing one boundary, all other inputs were kept at nominal valid values to isolate the effect of the boundary being tested.

Step 4: Review boundary coverage. The completed test suite was reviewed to verify that all applicable boundaries had been tested and that inputs without measurable boundaries were correctly excluded from Boundary Value Analysis. Additional test cases were added where necessary to ensure thorough boundary coverage.

I.Pool A - FR-03: Forgot password and password reset (two steps)

1. Requirement Analysis

Functional Requirements Summary:
FR-03 defines a two-step password reset flow:
Step 1: The user enters an already registered email address. The system generates a 6-digit OTP and presents it in the demo environment.
Step 2: The user enters the OTP, a new password, and a confirmation of the new password. The new password must follow the same strength rules as FR-01. The two password fields must match. The OTP must be valid for the same email that initiated the reset.

User Inputs:
- Email address (Step 1)
- OTP (Step 2)
- New password (Step 2)
- Confirm new password (Step 2)

Assumptions:
- The new password uses the same strength rules defined in FR-01: minimum 8 characters, at least one uppercase, one lowercase, one digit, and one special character.
- OTP is a numeric 6-digit value, represented as a string in the API.

- OTP validity is tied to the specific email that requested the reset.
- The spec does not explicitly define an expiry window or one-time-use rule, but SEC-07 suggests these should be treated as expected constraints.

2. Input Domain Identification

Input: Email address

- Data type: String
- Valid domain: A syntactically valid email address that belongs to an existing registered user
- Invalid domain: Empty, malformed, unregistered, non-string values
- Constraints: Must be present, follow email format, correspond to a registered account
- Dependencies: Existing user database, reset request initiated for that same email
- Assumptions: The UI uses email input semantics and the backend validates format

Input: OTP

- Data type: String
- Valid domain: A 6-digit numeric code that matches the OTP generated for the current reset request
- Invalid domain: Wrong value, wrong length, non-numeric, expired/used, empty
- Constraints: Must be exactly 6 digits, associated with the current email reset request
- Dependencies: The email that initiated reset, OTP generation record/session state
- Assumptions: OTP is single-use and time-bounded, as implied by security requirements

Input: New password

- Data type: String
- Valid domain: A password that meets the strength rules and is not empty
- Invalid domain: Too short, missing required character classes, empty/whitespace, non-string
- Constraints: Minimum 8 characters, at least one uppercase, one lowercase, one digit, one special character
- Dependencies: Password confirmation field, reset request context
- Assumptions: Same rules from FR-01 apply

Input: Confirm new password

- Data type: String
- Valid domain: A value identical to the new password
- Invalid domain: Empty, different from the new password, non-string
- Constraints: Must match the new password exactly
- Dependencies: New password input
- Assumptions: Confirmation is only a matching check, not an independent strength check

3. Equivalence Class Partitioning

Input: Email address

- EC1: Valid registered email, e.g. user@domain.com
- EC2: Valid format but unregistered email
- EC3: Empty or whitespace-only input
- EC4: Malformed email format, such as missing @, missing domain, invalid characters
- EC5: Wrong data type, such as null or numeric input

Why these classes exist: They separate successful flows from different failure categories and distinguish syntax problems from account-state problems.

Input: OTP

- EC1: Correct 6-digit OTP for the current reset request
- EC2: Incorrect 6-digit OTP for the same email
- EC3: OTP with fewer than 6 digits
- EC4: OTP with more than 6 digits
- EC5: Non-numeric OTP
- EC6: Empty OTP
- EC7: OTP that is expired or already consumed

Why these classes exist: They cover normal valid input and all meaningful invalid input patterns related to length, format, and validity state.

Input: New password

- EC1: Strong password meeting all rules
- EC2: Password shorter than 8 characters
- EC3: Password missing uppercase letter
- EC4: Password missing lowercase letter
- EC5: Password missing digit
- EC6: Password missing special character
- EC7: Empty or whitespace-only password
- EC8: Non-string input

Why these classes exist: Each class isolates a single rule violation, which is important for domain-driven validation.

Input: Confirm new password

- EC1: Matches the new password exactly
- EC2: Empty or whitespace-only confirmation
- EC3: Different from the new password
- EC4: Non-string input

4. Test cases for domain testing

Test Case ID	Requirement	Input(s)	Test Data	Equivalence Class ID	Expected Result	Test Type
TC-FP-001	Step 1: Submit a registered email to request OTP	Email address	user@domain.com	EC1	System accepts the email, generates a 6-digit OTP, and moves to Step 2	Valid
TC-FP-002	Step 1: Submit a syntactically valid but unregistered email	Email address	unknownuser@domain.com	EC2	System rejects the request with an appropriate error message and does not continue the reset flow	Invalid
TC-FP-003	Step 1: Submit an empty email field	Email address	""	EC3	System rejects the request and prompts for an email	Invalid
TC-FP-004	Step 1: Submit whitespace-only email	Email address	" "	EC3	System rejects the input as invalid	Invalid
TC-FP-005	Step 1: Submit malformed email format	Email address	invalid-email	EC4	System rejects the input with a format validation error	Invalid

TC-FP-006	Step 1: Submit null email value	Email address	null	EC5	System rejects the request and shows a validation error	Invalid
TC-FP-007	Step 1: Submit non-string email input	Email address	12345	EC5	System rejects the input and does not proceed	Invalid
TC-FP-008	Step 1: Submit a valid email with special characters in the local part or domain	Email address	user+tag@sub.domain.com	EC1	System accepts the email if it is syntactically valid and linked to a registered account	Valid
TC-FP-009	Step 2: Submit valid OTP and matching password fields	OTP, New password, Confirm new password	OTP: 123456; New password: StrongPass1!; Confirm: StrongPass1!	OTP EC1, Password EC1, Confirm EC1	Password reset succeeds and the user is notified of success	Valid
TC-FP-010	Step 2: Submit an incorrect OTP	OTP	999999	OTP EC2	System rejects the reset and shows an OTP validation error	Invalid
TC-FP-011	Step 2: Submit OTP shorter than 6 digits	OTP	12345	OTP EC3	System rejects the OTP as too short	Invalid
TC-FP-012	Step 2: Submit OTP	OTP	1234567	OTP EC4	System rejects the	Invalid

	longer than 6 digits				OTP as too long	
TC-FP-013	Step 2: Submit non-numeric OTP	OTP	ABC123	OTP EC5	System rejects the OTP as invalid format	Invalid
TC-FP-014	Step 2: Submit empty OTP	OTP	""	OTP EC6	System rejects the submission and requires OTP entry	Invalid
TC-FP-015	Step 2: Submit an expired or already used OTP	OTP	Previously expired/consumed OTP	OTP EC7	System rejects the reset and states that the OTP is no longer valid	Invalid
TC-FP-016	Step 2: Submit OTP that belongs to a different email context	OTP + email context	OTP generated for another email address	Dependency validation	System rejects the reset because the OTP is not valid for the current email	Invalid
TC-FP-017	Step 2: Submit a new password shorter than 8 characters	New password	Ab1!	Password EC2	System rejects the password as too short	Invalid
TC-FP-018	Step 2: Submit password missing	New password	strongpass1!	Password EC3	System rejects the password because it lacks an	Invalid

	uppercase letter				uppercase character	
TC-FP-019	Step 2: Submit password missing lowercase letter	New password	STRONGPASS1!	Password EC4	System rejects the password because it lacks a lowercase character	Invalid
TC-FP-020	Step 2: Submit password missing digit	New password	StrongPass!	Password EC5	System rejects the password because it lacks a numeric character	Invalid
TC-FP-021	Step 2: Submit password missing special character	New password	StrongPass1	Password EC6	System rejects the password because it lacks a special character	Invalid
TC-FP-022	Step 2: Submit empty new password	New password	""	Password EC7	System rejects the submission and requires a password	Invalid
TC-FP-023	Step 2: Submit whitespace-only new password	New password	" "	Password EC7	System rejects the password as invalid	Invalid
TC-FP-024	Step 2: Submit null new password	New password	null	Password EC8	System rejects the password and shows	Invalid

					a validation error	
TC-FP-025	Step 2: Submit empty confirm password	Confirm new password	""	Confirm EC2	System rejects the submission and asks the user to confirm the password	Invalid
TC-FP-026	Step 2: Submit confirm password that does not match the new password	Confirm new password	New password: StrongPass 1!; Confirm: StrongPass 2!	Confirm EC3	System rejects the reset and reports that the password values do not match	Invalid
TC-FP-027	Step 2: Submit non-string confirm password	Confirm new password	123456	Confirm EC4	System rejects the input and prevents reset	Invalid
TC-FP-028	Step 2: Submit multiple invalid inputs together	OTP, New password, Confirm new password	OTP: 000000; New password: weak; Confirm: different	OTP EC2, Password EC2/EC3/EC6, Confirm EC3	System reports all relevant validation errors and does not reset the password	Invalid
TC-FP-029	Step 2: Submit form with a missing required field	OTP, New password, Confirm new password	OTP omitted; New password provided; Confirm provided	OTP EC6	System blocks submission and highlights the missing required OTP field	Invalid
TC-FP-030	Step 2: Submit	New password,	New: P@ssw0rd!;	Password EC1,	System accepts the	Valid

	password with special characters and matching confirmation	Confirm new password	Confirm: P@ssw0rd!	Confirm EC1	password and allows password reset	

5. Boundary Identification

Input name	Validation rules	Min value / min length	Max value / max length	Range constraints	Assumptions
Email address	Valid email format; registered email	Usually 1 character minimum, but effectively defined by email syntax	None defined	Must be syntactically valid email	No explicit length boundaries in spec
OTP code	6-digit numeric OTP	6 digits	6 digits	Exactly 6 digits	OTP is a fixed-length token
New password	Must satisfy FR-01 strong password rules	Minimum 8 characters	None defined	>= 8 characters; must contain uppercase, lowercase, digit, special char	No explicit upper bound specified
Confirm password	Must match new password	Same as new password	Same as new password	Must be equal to new password	Confirmation is compared exactly

6. Boundary Value Analysis

Input	Just below minimum	Minimum	Just above minimum	Nominal value	Just below maximum	Maximum	Just above maximum	Why test?
Email address	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Email has no

								explicit measurable length boundaries in FR-03; BVA is not applicable.
OTP code	5 digits	6 digits	7 digits	6 digits with valid numeric value	N/A	N/A	N/A	Verify exact-length requirement of OTP and reject incorrect-length tokens.
New password length	7 chars	8 chars	9 chars	12 chars	N/A	N/A	N/A	Verify the minimum password length from FR-01 and the first valid longer password.
Confirm password	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Confirm password is a match check, not a measurable range

				input. BVA is not applicable beyond matching validation.

7. Test cases for boundary value analysis

Test Case ID	Input(s)	Test Data	Boundary Tested	Expected Result
BVA-FR03-01	OTP code; New password; Confirm password	OTP=123456; New password=Abcde1!2; Confirm password=Abcde1!2	OTP minimum valid length; Password minimum valid length	Reset flow accepts the input and proceeds with valid OTP and password.
BVA-FR03-02	OTP code; New password; Confirm password	OTP=12345; New password=Abcde1!2; Confirm password=Abcde1!2	OTP just below minimum	Reset flow rejects the OTP for being too short.
BVA-FR03-03	OTP code; New password; Confirm password	OTP=1234567; New password=Abcde1!2; Confirm password=Abcde1!2	OTP just above minimum	Reset flow rejects the OTP for being too long.
BVA-FR03-04	OTP code; New password; Confirm password	OTP=123456; New password=Abcde1!; Confirm password=Abcde1!	Password just below minimum length	Reset flow rejects the password for being too short.

BVA-FR03-05	OTP code; New password; Confirm password	OTP=123456; New password=Abcde1!23; Confirm password=Abcde1!23	Password just above minimum length	Reset flow accepts the password and proceeds.
BVA-FR03-06	OTP code; New password; Confirm password	OTP=123456; New password=Abcdef1!2345; Confirm password=Abcdef1!2345	Nominal password length	Reset flow accepts the password and proceeds with a normal valid-length password.

II. Pool B: FR-11: Order History View(User)

1. Requirement Analysis

Functional Requirements Summary:
FR-11 requires the user order history page to show only the authenticated user's own orders. The page must display each order's ID, order date, total amount, and current status. The status must be presented in Vietnamese and with distinct color coding.

User Inputs:
- Authenticated user identity / user context
- Request to load order history for the authenticated user
- Implicit backend order record data for the current user: Order ID, Order date, Total amount, Order status

Assumptions:
- The feature is a protected endpoint requiring authentication.
- There is no explicit pagination or filter input described, so the simplest view returns the user's available history.
- Order record fields are treated as input data to the view logic, even though they originate from storage rather than direct user entry.
- Status translation and color rendering are UI requirements and are not treated as additional inputs for this analysis.

2. Input Domain Identification

Input: Authenticated user identity
- Data type: Authentication context / JWT token
- Purpose: Determine which user's order history may be returned

- Valid domain: Logged-in user with valid token and user ID
- Invalid domain: Unauthenticated request, expired token, invalid token
- Constraints: Must be a valid authentication context
- Dependencies: Depends on session/token validation and user authorization
- Assumptions: Backend enforces 'user only sees own orders'

Input: Order history request
- Data type: API request context
- Purpose: Trigger retrieval of the current user's orders
- Valid domain: Well-formed request from authenticated user
- Invalid domain: Missing auth context, malformed request headers
- Constraints: Must include auth context and target endpoint
- Dependencies: Depends on authenticated user identity
- Assumptions: No request body is required for basic history retrieval

Input: Order ID
- Data type: Identifier (integer/string)
- Purpose: Identify each order record in the history
- Valid domain: Existing order identifier belonging to the authenticated user
- Invalid domain: Non-existing order ID, order ID belonging to another user
- Constraints: Must map to a valid order record for the current user
- Dependencies: Depends on authenticated user context and data ownership
- Assumptions: Order IDs are unique and stored correctly

Input: Order date
- Data type: Date/time
- Purpose: Display when the order was placed
- Valid domain: Valid date/time values
- Invalid domain: Missing, null, malformed date values
- Constraints: Should be a valid timestamp
- Dependencies: Depends on the order record
- Assumptions: Stored order dates are valid and retrievable

Input: Total amount
- Data type: Numeric / monetary
- Purpose: Display the order's total price
- Valid domain: Non-negative numeric amounts
- Invalid domain: Null, missing, non-numeric, negative amounts
- Constraints: Must represent a valid monetary value
- Dependencies: Depends on the order record

• Assumptions: Order totals are stored correctly and are >= 0

Input: Order status
• Data type: String
• Purpose: Display the current lifecycle stage of the order
• Valid domain: pending, confirmed, shipping, delivered, canceled
• Invalid domain: Unknown status, unsupported string, null
• Constraints: Must be a recognized order status
• Dependencies: Depends on the order record
• Assumptions: Stored status values are consistent with the state machine

3. Equivalence Class Partitioning

Input: Authenticated user identity
• EC1: Authenticated user with valid token (Valid) - Exists because only a valid user may access the history page.
• EC2: Unauthenticated request (Invalid) - Covers access denial when no user identity is provided.
• EC3: Invalid or expired token (Invalid) - Covers requests with invalid authentication context.

Input: Order history request
• EC4: Valid request from authenticated user (Valid) - Represents a normal history retrieval.

Input: Order ID
• EC5: Existing order ID for current user (Valid) - Represents a legitimate order displayed in the user's history.
• EC6: Existing order ID belonging to another user (Invalid) - Ensures user isolation is enforced.

Input: Order date
• EC7: Valid date/time (Valid) - Represents correct order timestamp display.

Input: Total amount
• EC8: Non-negative numeric amount (Valid) - Represents a correct monetary value.

Input: Order status
• EC9: Recognized valid status value (Valid) - Represents a proper state to display in the history.

4. Test cases for domain testing

Test Case ID	Requirement	Input(s)	Test Data	Equivalence Class ID	Expected Result	Test Type
DT-FR11-01	Valid user can view own order history	Authenticated user identity; Order history request; Order record fields	Valid JWT token for user A; history request to /api/orders/my-orders; order records include IDs, dates, totals, valid statuses for user A	EC1, EC4, EC5, EC7, EC8, EC9	User A sees only their orders with correct ID, date, total amount, and translated status with different colors.	Valid
DT-FR11-02	Empty order history acceptable	Authenticated user identity; Order history request	Valid JWT token for user B; history request; no orders exist for user B	EC1, EC4	System returns an empty order history list without error.	Valid
DT-FR11-03	Unauthenticated access is rejected	Authenticated user identity	No token	EC2	System rejects the request with an unauthorized error.	Invalid
DT-FR11-04	Expired or invalid token is rejected	Authenticated user identity	Expired JWT token	EC3	System rejects the request and returns authentication failure.	Invalid
DT-FR11-05	User cannot see another user's order	Order ID / order ownership	Valid token for user A; order record	EC6	System does not return user B's order for	Invalid

			belongs to user B		user A; access is denied or filtered out.	
DT-FR11-09	Missing auth context in request is rejected	Order history request	Request to /api/orders/my-orders without auth header	EC2	System rejects the request as missing required authentication context.	Invalid

5. Boundary Identification

Input name	Validation rules	Min value	Max value	Range constraints	Assumptions
Number of delivered orders	Must be non-negative	0	None defined	>= 0	Count derived from stored orders
Number of pending orders	Must be non-negative	0	None defined	>= 0	Count derived from stored orders
Number of confirmed orders	Must be non-negative	0	None defined	>= 0	Count derived from stored orders
Number of shipping orders	Must be non-negative	0	None defined	>= 0	Count derived from stored orders
Number of canceled orders	Must be non-negative	0	None defined	>= 0	Count derived from stored orders
Total amount per order	Must be non-negative	0	None defined	>= 0	Stored order totals are valid monetary values

Order date/time	Should be valid timestamp	N/A	N/A	N/A	No explicit date range defined in FR-11
Authenticated user identity	Must be authenticated	N/A	N/A	N/A	Not subject to BVA

6. Boundary Value Analysis

Input	Just below minimum	Minimum	Just above minimum	Nominal value	Just below maximum	Maximum	Just above maximum	Why test?
Number of delivered orders	-1	0	1	10	N/A	N/A	N/A	Verifies empty delivered history and the first valid delivered order count.
Number of pending orders	-1	0	1	10	N/A	N/A	N/A	Verifies zero pending orders and correct counting when pending orders exist.
Number of confirmed orders	-1	0	1	10	N/A	N/A	N/A	Verifies the lower boundary for confirme

								d order counts.
Number of shipping orders	-1	0	1	10	N/A	N/A	N/A	Verifies the lower boundary for shipping order counts.
Number of canceled orders	-1	0	1	10	N/A	N/A	N/A	Verifies the lower boundary for canceled order counts.
Total amount per order	-1	0	1	500000	N/A	N/A	N/A	Verifies zero-value orders and the first positive order amount.
Order date/time	N/A	N/A	N/A	N/A	N/A	N/A	N/A	No explicit date boundaries are defined, so BVA is not applicable for date/time.
Authenticated	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Categorical/authenticatio

user identity				n data; not applicable for BVA.

7. Test cases for boundary analysis

Test Case ID	Input(s)	Test Data	Boundary Tested	Expected Result
BVA-FR11-01	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders; Total amount per order	delivered=0, pending=0, confirmed=0, shipping=0, canceled=0, total_amount=0	Minimum for all numeric inputs	The history view handles the empty-order scenario correctly: all counts are 0 and total amounts are shown as 0.
BVA-FR11-02	Number of delivered orders; Total amount per order	delivered=1, pending=0, confirmed=0, shipping=0, canceled=0, total_amount=1	Just above minimum for delivered count and order amount	The history view correctly displays one delivered order and a positive order amount of 1.
BVA-FR11-03	Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders	delivered=0, pending=1, confirmed=1, shipping=1, canceled=1, total_amount=0	Just above minimum for all non-delivered status counts	The history view correctly counts one order for each non-delivered status and shows revenue/remains consistent with no delivered orders.
BVA-FR11-04	Total amount per order	delivered=0, pending=0, confirmed=0, shipping=0, canceled=0,	Nominal order amount with minimum counts	The history view accepts a large valid monetary amount and displays it correctly even

		total_amount=500 000		when order counts are at minimum.
BVA-FR11-05	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders; Total amount per order	delivered=10, pending=10, confirmed=10, shipping=10, canceled=10, total_amount=500 000	Nominal values for counts and amount	The history view handles normal-sized metric values correctly and displays expected totals for counts and amount.
BVA-FR11-06	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders; Total amount per order	delivered=1, pending=1, confirmed=1, shipping=1, canceled=1, total_amount=1	Just above minimum for all counts and order amount	The history view correctly handles a minimal non-zero set of orders across all statuses with the smallest positive order amount.

III. Pool C: FR-12: Dashboard

1. Requirement Analysis

Functional Requirements Summary:
FR-13 defines the admin dashboard metrics:
The dashboard must display total revenue.
Total revenue must include only orders whose status is delivered.
The dashboard must also display total number of orders.

User Inputs:
- Order data from the system/database
- Each order’s status
- Each order’s total_amount
- Admin access context

Assumptions:
- The dashboard reads order records from persisted storage.
- The total number of orders counts all order records, regardless of status.
- Revenue is calculated only from orders where status equals delivered.
- Admin access is already governed by FR-12 and is assumed to be valid for dashboard access.

2. Input Domain Identification

Input: Order dataset
- Data type: Collection/list of order records
- Purpose: Source of data for both metrics
- Valid domain: A readable collection of order records
- Invalid domain: Missing
- Constraints: Must be interpretable as a list of order records
- Dependencies: Depends on backend/database availability and data retrieval
- Assumptions: The dashboard loads data from the backend

Input: Order status
- Data type: String
- Purpose: Determines whether an order contributes to revenue
- Valid domain: delivered, pending, confirmed, shipping, canceled
- Constraints: Must be one of the defined status values
- Dependencies: Depends on each order record in the dataset
- Assumptions: Status is stored as a string in the orders table, and order status is always valid ensured by backend(data retrieved from backend is correct 100%).

Input: Order total_amount
- Data type: Number/integer
- Purpose: Used to compute revenue
- Valid domain: Numeric monetary values
- Constraints: Must be a valid numeric amount
- Dependencies: Depends on each order record in the dataset
- Assumptions: The amount is stored in the database and should be treated as monetary data, and total_amount is always Numeric monetary values as ensured by backend(data retrieved from backend is correct 100%).

Input: Admin access context
- Data type: Authentication/authorization context
- Purpose: Confirms the user is authorized to view the dashboard
- Valid domain: Authenticated admin user
- Invalid domain: Non-admin, unauthenticated
- Constraints: Must satisfy admin authorization

• Dependencies: Depends on authentication and role data
• Assumptions: FR-12 governs access to admin features

3. Equivalence Class Partitioning

Input: Order dataset

• EC1: Non-empty collection of order records (Valid) - This is the normal case where the dashboard must compute metrics from existing orders.
• EC2: Missing dataset (Invalid) - The dashboard should not proceed with a missing data source.

Input: Order status

• EC3: Status = delivered (Valid) - This is the only status that should contribute to revenue.
• EC4: Status = pending / confirmed / shipping / canceled (Valid) - These are valid order states, but they should not contribute to revenue for FR-13.

Input: Order total_amount

• EC5: Numeric monetary value (Valid) - This is the normal case for revenue calculation.

Input: Admin access context

• EC6: Authenticated admin user (Valid) - This is the intended access context for the admin dashboard.
• EC7: Non-admin / unauthenticated / missing role (Invalid) - These contexts should not allow dashboard access.

4. Test cases for domain testing

Test Case ID	Requirement	Input(s)	Test Data	Equivalence Class ID	Expected Result	Test Type
DT-FR13-01	Display dashboard metrics for normal admin view	Order dataset, order status, total_amount, admin access	Dataset contains 5 orders: one delivered with amount 600000, and 4 orders of the rest of status with total_amount =100000;	EC1, EC3, EC4, EC5, EC6	Dashboard displays total revenue as 600000and total order count as 3.	Valid

			role = admin			
DT-FR13-02	Display dashboard with no orders	Order dataset, order status, total_amount, admin access	Dataset is empty; role = admin	EC2, EC6	Dashboard displays zero revenue and zero total order count.	Valid
DT-FR13-03	Include delivered orders in revenue	Order dataset, order status, total_amount	Dataset contains one order with status = delivered and amount = 300000	EC1, EC3, EC5	Revenue includes that order amount.	Valid
DT-FR13-04	Exclude non-delivered orders from revenue	Order dataset, order status, total_amount	Dataset contains one order with status = pending and amount = 300000	EC1, EC4, EC5	Revenue does not include that order amount, but total order count still includes it.	Valid
DT-FR13-5	Reject special-character role input	Admin access context	role != "admin"	EC7	System rejects the request as invalid authorization input.	Invalid
DT-FR13-6	Reject special-character role input	Admin access context	null	EC7	System rejects the request as invalid authorization input.	Invalid

5. Boundary Identification

Input name	Validation rules	Min value	Max value	Range constraints	Assumptions
Number of delivered orders	Must be non-negative	0	None defined	>= 0	Count derived from orders with status='delivered'
Number of pending,canceled,shipping orders	Must be non-negative	0	None defined	>= 0	Count derived from order records
total_amount per order	Must be non-negative	0	None defined	>= 0	Each order amount is assumed valid
Admin access context	Must be authenticated admin	N/A	N/A	N/A	Not subject to BVA

6. Boundary Value Analysis

Input	Just below minimum	Minimum	Just above minimum	Just below maximum	Maximum	Just above maximum	Why test?
Number of delivered orders	-1	0	1	N/A	N/A	N/A	Verifies that negative counts are rejected, zero delivered orders are handled correctly, and the first valid delivered order is

							counted accurately.
Number of pending, confirmed ,shipping, canceled orders	-1	0	1	N/A	N/A	N/A	Verifies that negative counts are rejected, zero orders are accepted, and the first valid order is counted correctly for each status.
total_amount per order	-1	0	1	N/A	N/A	N/A	Verifies that negative order amounts are rejected, zero-value orders are handled correctly (if allowed), and the smallest positive order amount is processed correctly.
Admin access context	N/A	N/A	N/A	N/A	N/A	N/A	BVA not applicable; this is categoric

				al/authentication data.

7. Test cases for boundary value analysis

Test Case ID	Input(s)	Test Data	Boundary Tested	Expected Result
BVA-FR13-01	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders;	delivered=0, pending=0, confirmed=0, shipping=0, canceled=0,	Minimum for all count and amount inputs	Dashboard handles the empty-order scenario correctly: all counts = 0 and total revenue = 0.
BVA-FR13-02	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; total_amount per order	delivered=1,pending=2, confirmed=2, shipping=2, canceled=2, order amount=1	Just above minimum for delivered order count and order amount	Dashboard displays 9 delivered order and revenue = 1.
BVA-FR13-03	Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders	pending=1, confirmed=1, shipping=1, canceled=1,	Just above minimum for each non-delivered status count	Dashboard counts one order in each non-delivered status correctly and total orders includes all 4, with revenue = 0.
BVA-FR13-04	Number of delivered orders; total_amount per order	delivered=1, order amount=0	Minimum order amount for a delivered order	Dashboard accepts a delivered order with amount 0 and

				reports revenue = 0.
BVA-FR13-05	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders; total_amount per order	delivered=2, pending=1, confirmed=1, shipping=1, canceled=1, order amount=100000	Nominal values for counts and order amount	Dashboard handles normal-sized metric values correctly and displays expected totals.
BVA-FR13-06	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders; total_amount per order	delivered=1, pending=1, confirmed=1, shipping=1, canceled=1, order amount=1	Just above minimum for counts and amount across multiple statuses	Dashboard handles a mixed-status dataset with the smallest positive amount and displays correct totals.

IV.Pool D - FR-mobile: Forgot password and password reset (two steps)

1. Requirement Analysis

Functional Requirements Summary:

FR-**mobile** defines a two-step password reset flow:
Step 1: The user enters an already registered email address. The system generates a 6-digit OTP and presents it in the demo environment.
Step 2: The user enters the OTP, a new password, and a confirmation of the new password. The new password must follow the same strength rules as FR-01. The two password fields must match. The OTP must be valid for the same email that initiated the reset.

User Inputs:
- Email address (Step 1)
- OTP (Step 2)

- New password (Step 2)
- Confirm new password (Step 2)

Assumptions:
- The new password uses the same strength rules defined in FR-01: minimum 8 characters, at least one uppercase, one lowercase, one digit, and one special character.
- OTP is a numeric 6-digit value, represented as a string in the API.
- OTP validity is tied to the specific email that requested the reset.
- The spec does not explicitly define an expiry window or one-time-use rule, but SEC-07 suggests these should be treated as expected constraints.

2. Input Domain Identification

Input: Email address
- Data type: String
- Valid domain: A syntactically valid email address that belongs to an existing registered user
- Invalid domain: Empty, malformed, unregistered, non-string values
- Constraints: Must be present, follow email format, correspond to a registered account
- Dependencies: Existing user database, reset request initiated for that same email
- Assumptions: The UI uses email input semantics and the backend validates format

Input: OTP
- Data type: String
- Valid domain: A 6-digit numeric code that matches the OTP generated for the current reset request
- Invalid domain: Wrong value, wrong length, non-numeric, expired/used, empty
- Constraints: Must be exactly 6 digits, associated with the current email reset request
- Dependencies: The email that initiated reset, OTP generation record/session state
- Assumptions: OTP is single-use and time-bounded, as implied by security requirements

Input: New password
- Data type: String
- Valid domain: A password that meets the strength rules and is not empty
- Invalid domain: Too short, missing required character classes, empty/whitespace, non-string
- Constraints: Minimum 8 characters, at least one uppercase, one lowercase, one digit, one special character
- Dependencies: Password confirmation field, reset request context
- Assumptions: Same rules from FR-01 apply

Input: Confirm new password
- Data type: String

- Valid domain: A value identical to the new password
- Invalid domain: Empty, different from the new password, non-string
- Constraints: Must match the new password exactly
- Dependencies: New password input
- Assumptions: Confirmation is only a matching check, not an independent strength check

3. Equivalence Class Partitioning

Input: Email address

- EC1: Valid registered email, e.g. user@domain.com
- EC2: Valid format but unregistered email
- EC3: Empty or whitespace-only input
- EC4: Malformed email format, such as missing @, missing domain, invalid characters
- EC5: Wrong data type, such as null or numeric input

Why these classes exist: They separate successful flows from different failure categories and distinguish syntax problems from account-state problems.

Input: OTP

- EC1: Correct 6-digit OTP for the current reset request
- EC2: Incorrect 6-digit OTP for the same email
- EC3: OTP with fewer than 6 digits
- EC4: OTP with more than 6 digits
- EC5: Non-numeric OTP
- EC6: Empty OTP
- EC7: OTP that is expired or already consumed

Why these classes exist: They cover normal valid input and all meaningful invalid input patterns related to length, format, and validity state.

Input: New password

- EC1: Strong password meeting all rules
- EC2: Password shorter than 8 characters
- EC3: Password missing uppercase letter
- EC4: Password missing lowercase letter
- EC5: Password missing digit
- EC6: Password missing special character
- EC7: Empty or whitespace-only password
- EC8: Non-string input

Why these classes exist: Each class isolates a single rule violation, which is important for domain-driven validation.

Input: Confirm new password

- EC1: Matches the new password exactly

- EC2: Empty or whitespace-only confirmation
- EC3: Different from the new password
- EC4: Non-string input

4. Test cases for domain testing

Test Case ID	Requirement	Input(s)	Test Data	Equivalence Class ID	Expected Result	Test Type
TC-FP-001	Step 1: Submit a registered email to request OTP	Email address	user@domain.com	EC1	System accepts the email, generates a 6-digit OTP, and moves to Step 2	Valid
TC-FP-002	Step 1: Submit a syntactically valid but unregistered email	Email address	unknownuser@domain.com	EC2	System rejects the request with an appropriate error message and does not continue the reset flow	Invalid
TC-FP-003	Step 1: Submit an empty email field	Email address	""	EC3	System rejects the request and prompts for an email	Invalid
TC-FP-004	Step 1: Submit whitespace-only email	Email address	" "	EC3	System rejects the input as invalid	Invalid
TC-FP-005	Step 1: Submit malformed	Email address	invalid-email	EC4	System rejects the input with a format	Invalid

	email format				validation error	
TC-FP-006	Step 1: Submit null email value	Email address	null	EC5	System rejects the request and shows a validation error	Invalid
TC-FP-007	Step 1: Submit non-string email input	Email address	12345	EC5	System rejects the input and does not proceed	Invalid
TC-FP-008	Step 1: Submit a valid email with special characters in the local part or domain	Email address	user+tag@sub.domain.com	EC1	System accepts the email if it is syntactically valid and linked to a registered account	Valid
TC-FP-009	Step 2: Submit valid OTP and matching password fields	OTP, New password, Confirm new password	OTP: 123456; New password: StrongPass1!; Confirm: StrongPass1!	OTP EC1, Password EC1, Confirm EC1	Password reset succeeds and the user is notified of success	Valid
TC-FP-010	Step 2: Submit an incorrect OTP	OTP	999999	OTP EC2	System rejects the reset and shows an OTP validation error	Invalid
TC-FP-011	Step 2: Submit OTP	OTP	12345	OTP EC3	System rejects the	Invalid

	shorter than 6 digits				OTP as too short	
TC-FP-012	Step 2: Submit OTP longer than 6 digits	OTP	1234567	OTP EC4	System rejects the OTP as too long	Invalid
TC-FP-013	Step 2: Submit non-numeric OTP	OTP	ABC123	OTP EC5	System rejects the OTP as invalid format	Invalid
TC-FP-014	Step 2: Submit empty OTP	OTP	""	OTP EC6	System rejects the submission and requires OTP entry	Invalid
TC-FP-015	Step 2: Submit an expired or already used OTP	OTP	Previously expired/consumed OTP	OTP EC7	System rejects the reset and states that the OTP is no longer valid	Invalid
TC-FP-016	Step 2: Submit OTP that belongs to a different email context	OTP + email context	OTP generated for another email address	Dependency validation	System rejects the reset because the OTP is not valid for the current email	Invalid
TC-FP-017	Step 2: Submit a new password shorter than 8 characters	New password	Ab1!	Password EC2	System rejects the password as too short	Invalid

TC-FP-018	Step 2: Submit password missing uppercase letter	New password	strongpass1!	Password EC3	System rejects the password because it lacks an uppercase character	Invalid
TC-FP-019	Step 2: Submit password missing lowercase letter	New password	STRONGPASS1!	Password EC4	System rejects the password because it lacks a lowercase character	Invalid
TC-FP-020	Step 2: Submit password missing digit	New password	StrongPass!	Password EC5	System rejects the password because it lacks a numeric character	Invalid
TC-FP-021	Step 2: Submit password missing special character	New password	StrongPass1	Password EC6	System rejects the password because it lacks a special character	Invalid
TC-FP-022	Step 2: Submit empty new password	New password	""	Password EC7	System rejects the submission and requires a password	Invalid
TC-FP-023	Step 2: Submit whitespace-only new password	New password	" "	Password EC7	System rejects the password as invalid	Invalid

TC-FP-024	Step 2: Submit null new password	New password	null	Password EC8	System rejects the password and shows a validation error	Invalid
TC-FP-025	Step 2: Submit empty confirm password	Confirm new password	""	Confirm EC2	System rejects the submission and asks the user to confirm the password	Invalid
TC-FP-026	Step 2: Submit confirm password that does not match the new password	Confirm new password	New password: StrongPass 1!; Confirm: StrongPass 2!	Confirm EC3	System rejects the reset and reports that the password values do not match	Invalid
TC-FP-027	Step 2: Submit non-string confirm password	Confirm new password	123456	Confirm EC4	System rejects the input and prevents reset	Invalid
TC-FP-028	Step 2: Submit multiple invalid inputs together	OTP, New password, Confirm new password	OTP: 000000; New password: weak; Confirm: different	OTP EC2, Password EC2/EC3/EC6, Confirm EC3	System reports all relevant validation errors and does not reset the password	Invalid
TC-FP-029	Step 2: Submit form with a missing required field	OTP, New password, Confirm new password	OTP omitted; New password provided; Confirm provided	OTP EC6	System blocks submission and highlights the missing	Invalid

					required OTP field	
TC-FP-030	Step 2: Submit password with special characters and matching confirmation	New password, Confirm new password	New: P@ssw0rd!; Confirm: P@ssw0rd!	Password EC1, Confirm EC1	System accepts the password and allows password reset	Valid

5. Boundary Identification

Input name	Validation rules	Min value / min length	Max value / max length	Range constraints	Assumptions
Email address	Valid email format; registered email	Usually 1 character minimum, but effectively defined by email syntax	None defined	Must be syntactically valid email	No explicit length boundaries in spec
OTP code	6-digit numeric OTP	6 digits	6 digits	Exactly 6 digits	OTP is a fixed-length token
New password	Must satisfy FR-01 strong password rules	Minimum 8 characters	None defined	>= 8 characters; must contain uppercase, lowercase, digit, special char	No explicit upper bound specified
Confirm password	Must match new password	Same as new password	Same as new password	Must be equal to new password	Confirmation is compared exactly

6. Boundary Value Analysis

Input	Just below	Minimum	Just above	Nominal value	Just below	Maximum	Just above	Why test?

	minimum		minimum		maximum		maximum	
Email address	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Email has no explicit measurable length boundaries in FR-03; BVA is not applicable.
OTP code	5 digits	6 digits	7 digits	6 digits with valid numeric value	N/A	N/A	N/A	Verify exact-length requirement of OTP and reject incorrect-length tokens.
New password length	7 chars	8 chars	9 chars	12 chars	N/A	N/A	N/A	Verify the minimum password length from FR-01 and the first valid longer password.

Confirm password	N/A	N/A	N/A	N/A	N/A	N/A	N/A	Confirm password is a match check, not a measurable range input. BVA is not applicable beyond matching validation.

7. Test cases for boundary value analysis

Test Case ID	Input(s)	Test Data	Boundary Tested	Expected Result
BVA-FR-Mobile-01	OTP code; New password; Confirm password	OTP=123456; New password=Abcde1!2; Confirm password=Abcde1!2	OTP minimum valid length; Password minimum valid length	Reset flow accepts the input and proceeds with valid OTP and password.
BVA-FR-Mobile-02	OTP code; New password; Confirm password	OTP=12345; New password=Abcde1!2; Confirm password=Abcde1!2	OTP just below minimum	Reset flow rejects the OTP for being too short.
BVA-FR-Mobile-03	OTP code; New password; Confirm password	OTP=1234567; New password=Abcde1!2; Confirm password=Abcde1!2	OTP just above minimum	Reset flow rejects the OTP for being too long.

BVA-FR-Mobile-04	OTP code; New password; Confirm password	OTP=123456; New password=Abcde1!; Confirm password=Abcde1!	Password just below minimum length	Reset flow rejects the password for being too short.
BVA-FR-Mobile-05	OTP code; New password; Confirm password	OTP=123456; New password=Abcde1!23; Confirm password=Abcde1!23	Password just above minimum length	Reset flow accepts the password and proceeds.
BVA-FR-Mobile-06	OTP code; New password; Confirm password	OTP=123456; New password=Abcdef1!2345; Confirm password=Abcdef1!2345	Nominal password length	Reset flow accepts the password and proceeds with a normal valid-length password.